

* File-Handling

* Assigning file pointer

```
FILE *fh_output;
```

```
fh_output = fopen ("io.txt", "w");
```

↓
similar

file
name

↓
mode in which
you are opening.

to open() in python.

* `fclose(fh_output);` // closes the file.

* `fputs("A string\n", fh_output)` // writes "A string"
// to the file.

`fputs` ↔ ~~fwrite~~ `write()` in python
similar.

`fopen` ↔ `open()` " "

`fclose` ↔ `close()` " "

* another function to write to files:

```
fprintf(fh_output, "data: (%d)\n", 5)
```

file pointer/
file handle

To be
written to
file

∴ It will be inputted as data:5.

Instead of 5, we can also give a variable.

* Different modes to open a file in C:

"w"

→ write mode

"a"

→ append mode

"r"

→ read mode

* How to read a file?

```
FILE *fh_input;
```

```
fh_input = fopen("io.txt", "r");
```

```
int finput = 0;
```

```
fscanf(fh_input, "%d", &finput);
```

file handle format
specifier

→ variable where
file contents
are stored.

* `fscanf` reads until it encounters a "`\n`"
ie, reads a single line ending with "`\n`".

* V can also use `fgetc()` to ^{read} ~~write~~ contents
from a file but again it reads only till
"`\n`".

~~Syntax:~~

```
char mystring[50];
```

```
fgetc(fh_input,
```

```
fgetc(finput
```

Syntax:

```
char mystring[50];
```

```
fgets(mystring, 50, fh_input);
```

max size
of data to read

variable, where
it should be
stored

file pointer

* if file opened for reading is empty;

fopen function will return NULL

* To read an entire file use "while loop"

```
while (fgets(mystring, 50, fh_input)) {
```

```
    printf("%s", mystring);
```

```
}
```

```
fclose(fh_input);
```

* Better practice:

→ check whether file is empty before opening & reading it using "if".

```
if (fh_input == NULL) { // fh_input → file ptr  
    printf("File is empty");  
}
```

* Both "w" & "a"; "w+" & "a+" will create a new file if file doesn't exist